

自然语言处理 (2025 春季学期)

二、自然语言处理基础



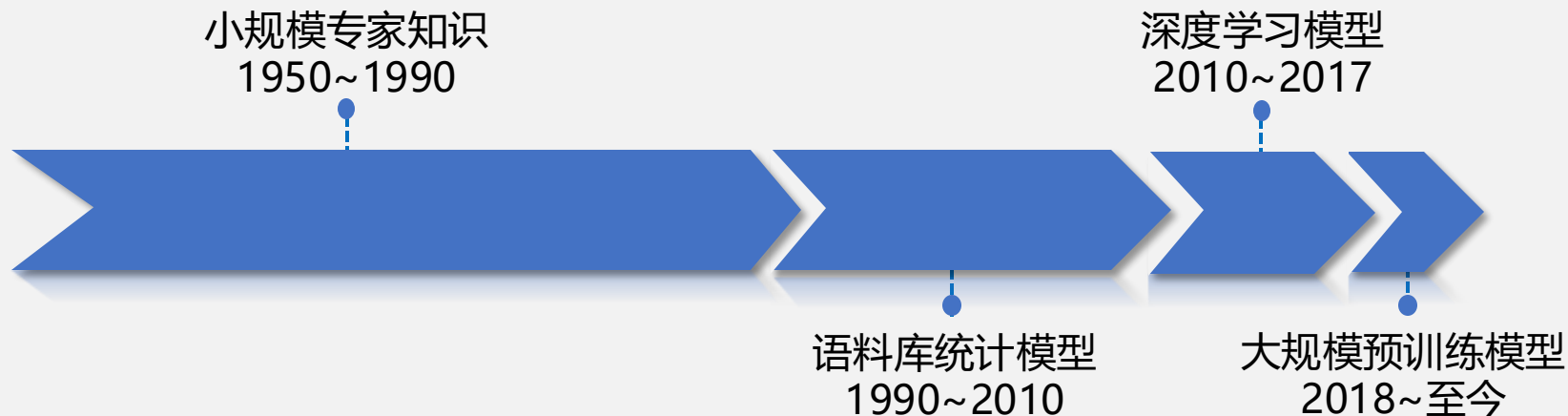
大纲

- 文本的表示
- 自然语言处理任务
- 自然语言处理的基本问题
- 自然语言处理的评价指标
- 基础工具集与常用数据集



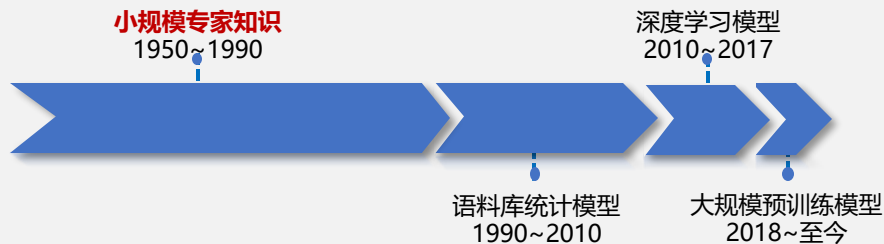
自然语言处理技术发展历史

- 语义在计算机内部是如何**表示**的？
- 根据表示方法的不同，自然语言处理技术经历了**四次范式变迁**





基于符号（字符串）表示的专家知识



□ “土豆非常好吃。”的情感倾向性？

- 如果：出现褒义词 “好” “喜欢” 等
- 那么：结果为褒义
- 如果：出现 “不”
- 那么：结果倾向性取反

□ 优点

- 符合人类的直觉
- 可解释、可干预性好

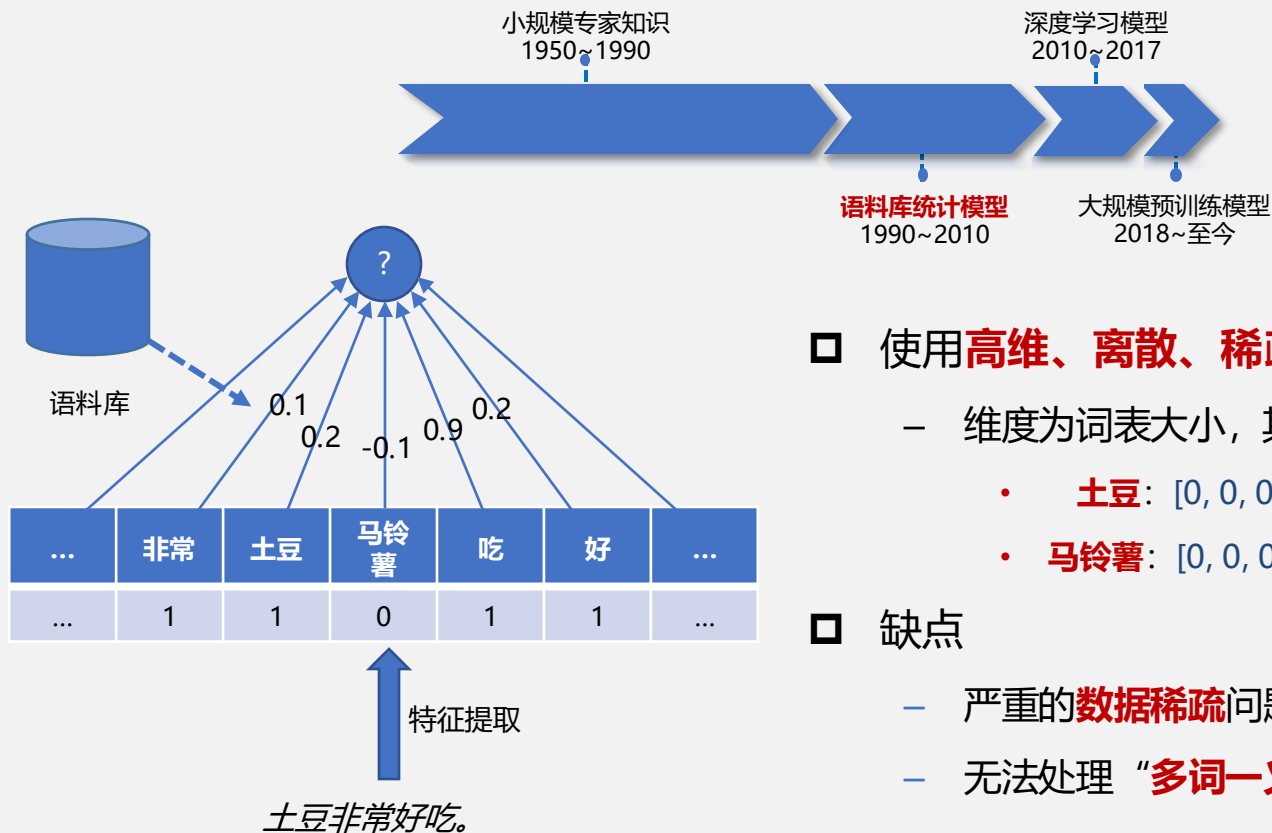
□ 缺点

- 知识完备性不足
- 需要专家构建和维护
- 不便于计算





基于向量表示的统计模型



□ 使用高维、离散、稀疏的向量表示词

— 维度为词表大小，其中只有一位为1，其余为0

• 土豆: [0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, ...]

• 马铃薯: [0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, ...]

□ 缺点

— 严重的数据稀疏问题

— 无法处理“多词一义”的现象



传统解决方案

□ 增加额外的特征

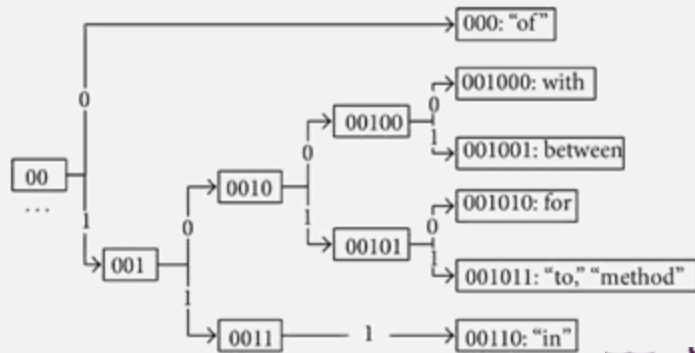
- 词性特征：名词、动词、形容词
- 前后缀特征：re-、-tion、-er

□ 语义词典

- WordNet、HowNet等
- 如词的上位信息表示语义类别
- 需要解决一词多义问题
- 收录的词不全且更新慢

□ 词聚类特征

- 如Brown Clustering (Brown et al., CL 1992)





词的分布语义假设

□ 分布语义假设 (Distributional semantic hypothesis)

– 词的含义可由其上下文词的分布进行表示

- *You shall know a word by the company it keeps* -- Firth J.R. 1957

he curtains open and the moon shining in on the barely
ars and the cold , close moon " . And neither of the w
rough the night with the moon shining so brightly , it
made in the light of the moon . It all boils down , wr
surely under a crescent moon , thrilled by ice-white
sun , the seasons of the moon ? Home , alone , Jay pla
m is dazzling snow , the moon has risen full and cold
un and the temple of the moon , driving out of the hug
in the dark and now the moon rises , full and amber a
bird on the shape of the moon over the trees in front
But I could n't see the moon or the stars , only the
rning , with a sliver of moon hanging among the stars
they love the sun , the moon and the stars . None of
the light of an enormous moon . The plash of flowing w
man 's first step on the moon ; various exhibits , aer
the inevitable piece of moon rock . Housing The Airsh
oud obscured part of the moon . The Allied guns behind



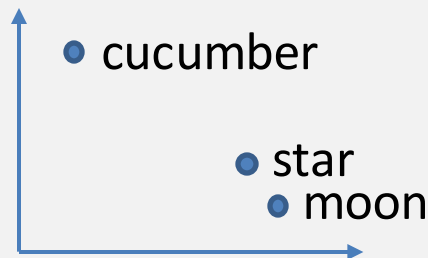


词的分布 (Distributional) 表示

□ 分布词向量

	shinning	bright	trees	dark	look
moon	38	45	2	27	12

□ 语义相似度通过计算向量相似度获得



□ 仍然存在高维、稀疏、离散的问题



分布表示的加权

□ 降低高频词的权重

— 点互信息 (Pointwise Mutual Information, PMI)

$$\text{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

我喜欢自然语言处理。
我爱深度学习。
我喜欢机器学习。



	我	喜欢	自然	语言	处理	爱	深度	学习	机器	。
我	0	2	1	1	1	1	1	2	1	3
喜欢	2	0	1	1	1	0	0	1	1	2
自然	1	1	0	1	1	0	0	0	0	1
语言	1	1	1	0	1	0	0	0	0	1
处理	1	1	1	1	0	0	0	0	0	1
爱	1	0	0	0	0	0	1	1	0	1
深度	1	0	0	0	0	1	0	1	0	1
学习	2	1	0	0	0	1	1	0	1	1
机器	1	1	0	0	0	0	0	1	0	1
。	3	2	1	1	1	1	1	2	1	0

```
def pmi(M, positive=True):
    col_totals = M.sum(axis=0) # 按列求和
    row_totals = M.sum(axis=1) # 按行求和
    total = col_totals.sum() # 总频次
    expected = np.outer(row_totals, col_totals) / total # 获得每个元素的分子
    M = M / expected

    with np.errstate(divide='ignore'): # 不显示log(0)的警告:
        M = np.log(M)
    M[np.isinf(M)] = 0.0 # 将log(0)置为0
    if positive:
        M[M < 0] = 0.0
    return M
```

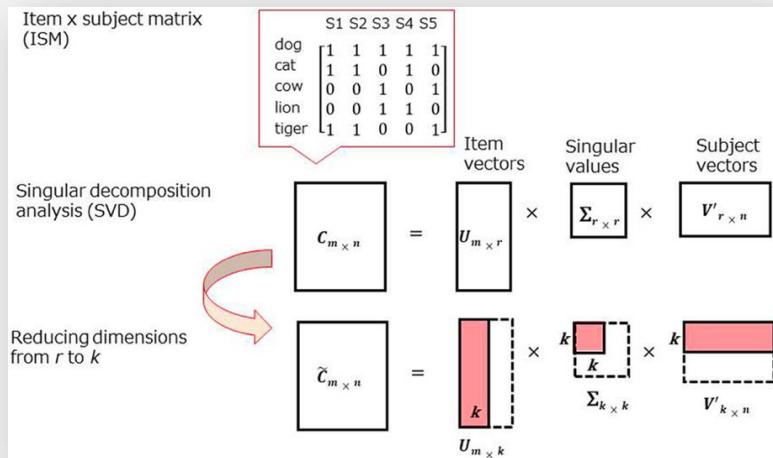
```
[[0.  0.17 0.06 0.06 0.06 0.28 0.28 0.28 0.28 0.28]
 [0.17 0.  0.43 0.43 0.43 0.  0.  0.  0.65 0.25]
 [0.06 0.43 0.  1.02 1.02 0.  0.  0.  0.  0.14]
 [0.06 0.43 1.02 0.  1.02 0.  0.  0.  0.  0.14]
 [0.06 0.43 1.02 1.02 0.  0.  0.  0.  0.  0.14]
 [0.28 0.  0.  0.  0.  0.  1.46 0.77 0.  0.36]
 [0.28 0.  0.  0.  0.  1.46 0.  0.77 0.  0.36]
 [0.42 0.09 0.  0.  0.  0.9 0.9 0.  0.9 0.  ]
 [0.28 0.65 0.  0.  0.  0.  0.  0.77 0.  0.36]
 [0.2  0.17 0.06 0.06 0.06 0.28 0.28 0.28 0.28 0.  ]]
```



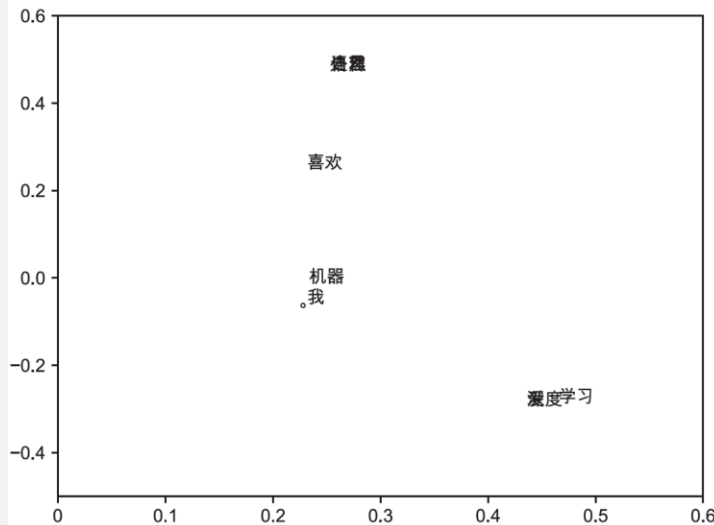
分布表示的降维

□ 避免稀疏性，反映高阶共现关系

– 奇异值分解 (Singular Value Decomposition, SVD)



```
U, s, Vh = np.linalg.svd(M_pmi)
```





分布式 (Distributed) 词表示

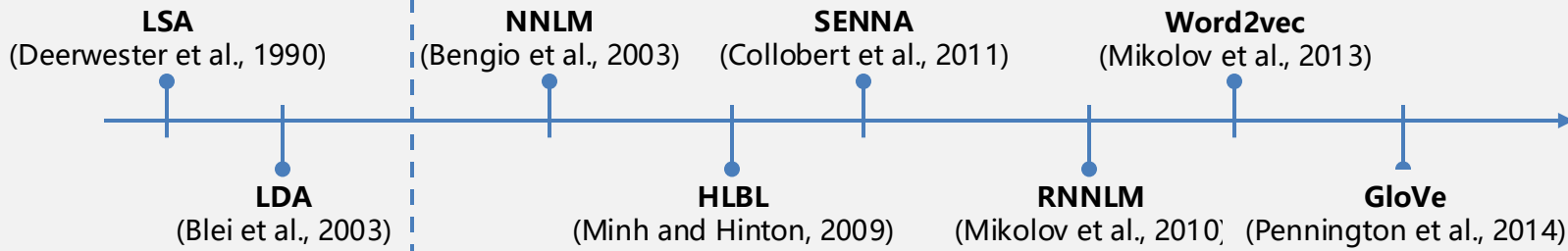
□ 分布表示的缺点

- 训练**速度慢**，增加**新语料库**困难
- **不易扩展**到短语、句子表示

□ 分布式表示**直接使用**低维、稠密、连续的向量表示词

- 通过 “**自监督**” 的方法直接学习词向量
- 也称**词嵌入** (Word Embedding)

□ 发展历程 (后续将详细介绍)





大纲

- 文本的表示
- 自然语言处理任务
- 自然语言处理的基本问题
- 自然语言处理的评价指标
- 基础工具集与常用数据集



语言模型

□ 语言模型 (Language Model, LM)

- 描述一段自然语言的概率或给定上文时下一个词出现的概率
 - $P(w_1, \dots w_l), P(w_{l+1}|w_1, \dots w_l)$.
- 以上两种定义等价 (链式法则)

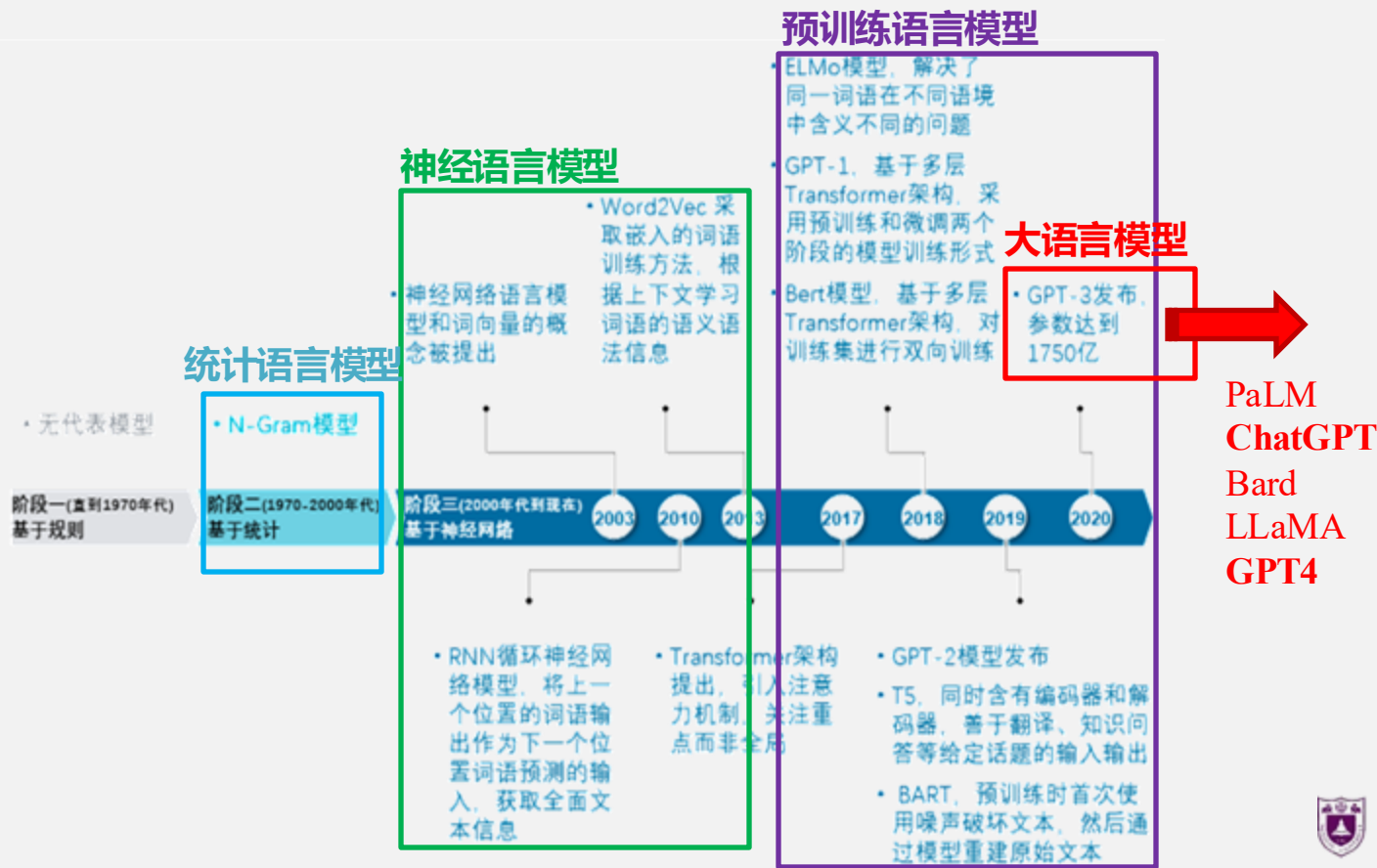
$$\begin{aligned} P(w_1 w_2 \dots w_l) &= P(w_1) P(w_2|w_1) P(w_3|w_1 w_2) \dots P(w_l|w_1 w_2 \dots w_{l-1}) \\ &= \prod_{i=1}^l P(w_i|w_{1:i-1}) \end{aligned}$$

□ 广泛应用于多种自然语言处理任务

- 机器翻译 (词排序)
 - $P(\text{the cat is small}) > P(\text{small the is cat})$
- 语音识别 (词选择)
 - $P(\text{there are four cats}) > P(\text{there are for cats})$



自然语言模型的发展历史





中文分词

□ 词 (Word)

- 最小的能独立使用的音义结合体
- 以汉语为代表的汉藏语系，以阿拉伯语为代表的闪-含语系中不包含明显的词之间的分隔符

□ 中文分词是将中文字序列切分成一个个单独的词

□ 分词的歧义

- 如：严守一把手机关了
 - 严守一/ 把/ 手机/ 关/ 了
 - 严守/ 一把手/ 机关/ 了
 - 严守/ 一把/ 手机/ 关/ 了
 - 严守一/ 把手/ 机关/ 了
 -



英文分词

□ 以**英语**为代表的印欧语系语言，是否需要进行分词？

□ 这些语言**词形变化**复杂

– 如： *computer*、*computers*、*computing*等

□ 用**空格**切分

– 优点：语义明确

– 缺点：数据稀疏，词表过大，降低处理速度，OOV

• sentence: I enjoy watching movies and reading books.

• result: I | enjoy | watching | movies | and | reading | books | .

□ 按字母切分

– 优点：词表极小，无OOV问题

– 缺点：语义信息不明确、编码解码效率低

• sentence: I enjoy watching movies and reading books.

• result: I | | e | n | j | o | y | | w | a | t | c | h | i | |





子词切分 (Subword)

□ 子词切分

- sentence: I enjoy watching movies and reading books.
- result: I | enjoy | watch | ing | movie | s | and | read | ing | book | s | .

- 折中：将一个单词切分为若干连续的**片段**（子词）
- 优点：语义明确，解决OOV问题，词表规模适中，编码解码效率高，多语言适用性
- 方法众多，**基本原理相似**
 - 使用尽量长且频次高的子词对单词进行切分

□ 目前主流大模型采用的基于子词的分词算法：

- **BPE (Byte-Pair Encoding)** : GPT-1, RoBERTa
- **BBPE (Byte-level BPE)** : LLaMA, ChatGLM, Bloom, ChatGPT, GPT-4
- WordPiece : BERT, DistilBERT, MobileBERT
- Unigram : ALBERT, T5, mBART, XLNet



字节对编码 (BPE)

Algorithm 1: BPE 中子词词表构造算法

Input: 大规模生文本语料库; 期望的子词词表大小 L

Output: 子词词表

将语料库中每个单词切分成字符作为子词;

用切分的子词构成初始子词词表;

while 子词词表小于等于 L **do**

 在语料库中统计单词内相邻子词对的频次;

 选取频次最高的子词对, 合并成新的子词;

 将新的子词加入子词词表;

 将语料库中不再存在的子词从子词词表中删除;

end

语料库: {'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}

初始子词词表: {'l', 'o', 'w', 'e', 'r', '</w>', 'n', 's', 't', 'i', 'd'}

合并子词词表: {'l', 'o', 'w', 'e', 'r', '</w>', 'n', 't', 'i', 'd', 'es'}

语料库: {'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}

合并子词词表: {'l', 'o', 'w', 'e', 'r', '</w>', 'n', 'i', 'd', 'est'}

语料库: {'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}

子词词表构造示例





字节对编码 (BPE)

□ BPE子词切分算法

1. 将子词词表按照子词的长度由大到小进行排序
2. 从前向后遍历子词词表，依次判断一个子词是否为单词的子串
3. 如果是则将该单词进行切分，然后继续向后遍历子词词表
4. 如果子词词表全部遍历结束，单词中仍然有子串没有被切分，那么这些子串一定为低频串，则使用统一的标记，如'<UNK>'进行替换

□ 更多子词切分算法

- WordPiece
- Unigram Language Model (ULM)
- SentencePiece
 - <https://github.com/google/sentencepiece>



BBPE (Byte-level BPE)

Original		質問して__証明と証拠を求めましょう	Ask__questions,__demand__proof,__demand__evidence.
Byte		E8 B3 AA E5 95 8F E3 81 97 E3 81 A6 E2 96 81 E8 A8 BC E6 98 8E E3 81 A8 E8 A8 BC E6 8B A0 E3 82 92 E6 B1 82 E3 82 81 E3 81 BE E3 81 97 E3 82 87 E3 81 86	41 73 6B E2 96 81 71 75 65 73 74 69 6F 6E 73 2C E2 96 81 64 65 6D 61 6E 64 E2 96 81 70 72 6F 6F 66 2C E2 96 81 64 65 6D 61 6E 64 E2 96 81 65 76 69 64 65 6E 63 65 2E
BBPE	1K	E8 B3 AA E595 8F しE381 A6 __E8 A8 BC 明 E381 A8 E8 A8 BC E6 8B A0 をE6 B1 82 めE381 BE しょう	A s k __quest ions , __dem and __pro of , __dem and __ev idence .
	2K	E8 B3 AA 問 しE381 A6 __E8 A8BC 明 E381 A8 E8 A8BC E68B A0 を E6 B1 82 めE381 BE しょう	A s k __qu est ion s , __d em and __pro of , __d em and __e v idence .
	4K	E8 B3 AA 問 しE381 A6 __E8 A8BC 明E381 A8 E8 A8BC 抛 をE6 B1 82 めE381 BE しょう	As k __quest ions , __d em and __pro of , __d em and __ev idence .
	8K	E8 B3 AA問 しE381 A6 __E8 A8BC 明E381 A8 E8 A8BC 抛 をE6 B1 82めE381 BE しょう	As k __questions , __demand __pro of , __demand __evidence .
	16K	E8 B3 AA問 しE381 A6 __E8 A8BC 明E381 A8 E8 A8BC 抛 をE6 B1 82めE381 BE しょう	As k __questions , __demand __proof , __demand __evidence .
	32K	E8 B3 AA問しE381 A6 __E8 A8BC 明E381 A8 E8 A8BC 抛 をE6 B1 82 めE381 BE しょう	As k __questions , __demand __proof , __demand __evidence .
CHAR		質 問 し て __ 証 明 と 証 拠 を 求 め ま し ょ う	A s k __ q u e s t i o n s , __ d e m a n d __ p r o o f , __ d e m a n d __ e v i d e n c e .
BPE	16K	質問して __ 証明 と 証拠 を求めましょう	A s k __questions , __demand __pro of , __demand __evidence .
	32K	質問して __ 証明 と 証拠 を求めましょう	A s k __questions , __demand __proof , __demand __evidence .



句法分析

- ❑ 分析句子的句法成分，如主谓宾定状补等
- ❑ 将词序列表示的句子转换成树状结构



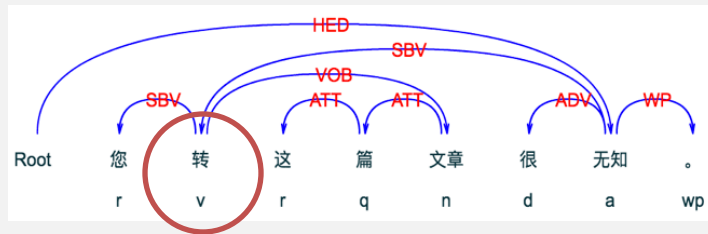
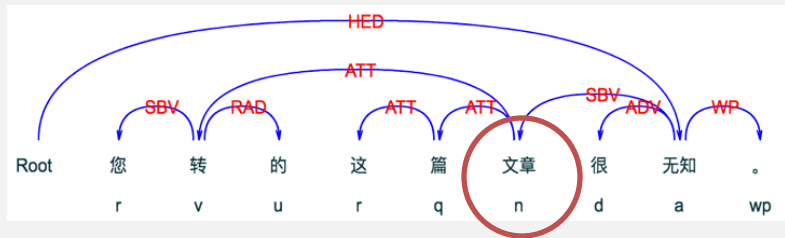
山寨发布会阳淼

@ [REDACTED] 才看到。昨天手机打字，把“您转的这篇文章很无知”打成了“您转这篇文章很无知”，少了一个的字。抱歉。



山寨发布会阳淼

主语是那篇文章很无知。



南京大學
NANJING UNIVERSITY

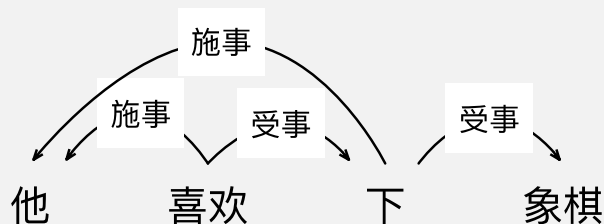


语义分析

- 词义消歧 (Word Sense Disambiguation, WSD)
- 语义角色标注 (Semantic Role Labeling, SRL)
 - 也称谓词论元结构 (Predicate-Argument Structure)

输入	他	喜欢	下	象棋	。
输出 1	施事	谓词		受事	
输出 2	施事		谓词	受事	

- 语义依存图 (Semantic Dependency Graph)





信息抽取

□ 信息抽取 (Information Extraction, IE)

- 从非结构化的文本中自动提取结构化信息

□ 输入

- 10月28日，AMD宣布斥资350亿美元收购FPGA芯片巨头赛灵思。这两家传了多年绯闻的芯片公司终于走到了一起。

信息抽取子任务	抽取结果
命名实体识别	公司名：AMD 公司名：赛灵思
关系抽取	赛灵思 $\xrightarrow{\text{从属}}$ AMD
时间表达式抽取	10月28日
时间表达式归一化	10月28日 $\rightarrow$ 2020年10月28日
事件抽取	事件：收购 时间：2020年10月28日 收购者：AMD 被收购者：赛灵思 收购金额：350亿美元



情感分析

□ 情感分析 (Sentiment Analysis)

- 个体对外界事物的态度、观点或倾向性，如正面、负面等
- 人自身的情绪 (Emotion)，如喜怒哀乐惧等

□ 输入

- 这款手机的屏幕很不错，性能也还可以。

情感分析子任务	分析结果
情感分类	褒义
情感信息抽取	评价词：不错；可以 评价对象：屏幕；性能 评价搭配：屏幕 ⇔ 不错；性能 ⇔ 可以





问答系统

□ 问答系统（Question Answering, QA）

- 用户以自然语言形式描述问题，从异构数据中获得答案

□ 根据数据来源的不同，问答系统可以分为4种主要的类型

– 检索式问答系统

- 答案来源于固定的文本语料库或互联网，系统通过查找相关文档并抽取答案完成问答

– 知识库问答系统

- 回答问题所需的知识以数据库等结构化形式存储，问答系统首先将问题解析为结构化的查询语句，通过查询相关知识点，并结合知识推理获取答案

– 常问问题集问答系统

- 通过对历史积累的常问问题集合进行检索，回答用户提出的类似问题

– 阅读理解式问答系统

- 通过抽取给定文档中的文本片段或生成一段答案来回答用户提出的问题



更多任务

- ❑ 机器翻译（Machine Translation, MT）
- ❑ 对话系统（Dialogue System）

	任务型 Task	聊天 Chat	知识问答 Knowledge	推荐 Recommendation
目的	完成任务或动作	闲聊	知识获取	信息推荐
领域	特定域（垂类）	开放域	开放域	特定域
以话轮数评价	越少越好	话轮越多越好	越少越好	越少越好
应用	虚拟个人助理	娱乐、情感陪护	客服、教育	个性化推荐
典型系统	Siri、Cortana、 Google Assistant、 度秘	小冰、笨笨	Watson、 Wolfram Alpha	阿里小蜜

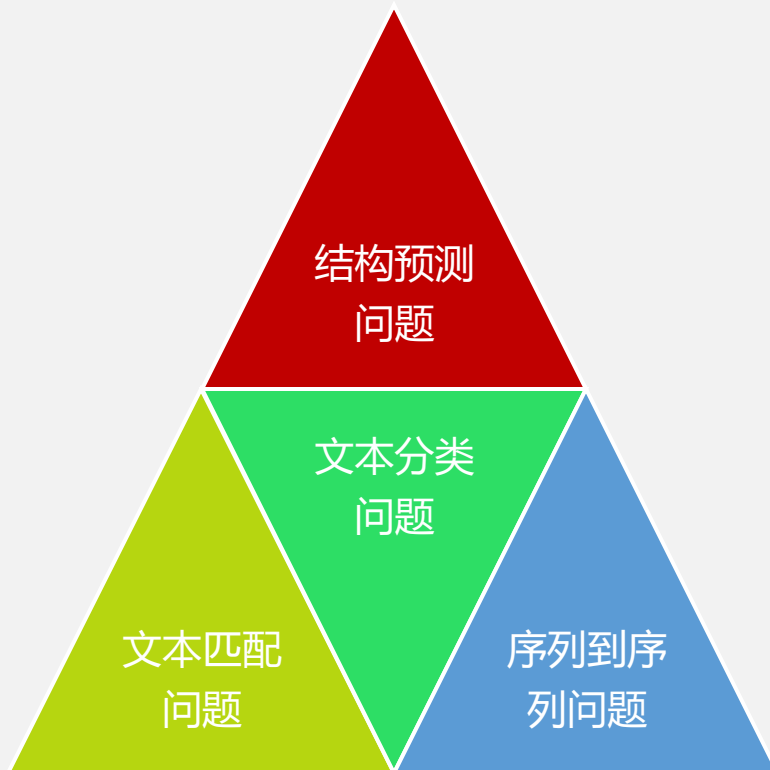


大纲

- 文本的表示
- 自然语言处理任务
- 自然语言处理的基本问题
- 自然语言处理的评价指标
- 基础工具集与常用数据集



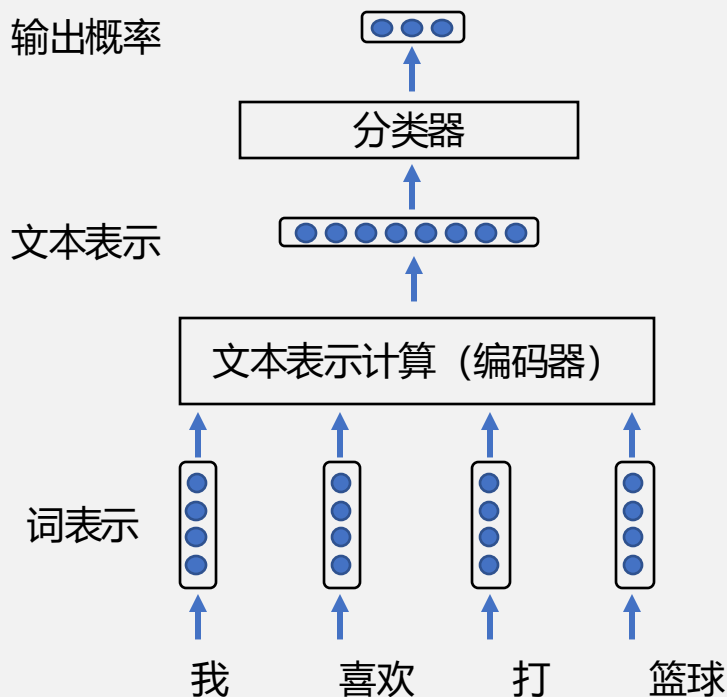
归结为四个基本问题





文本分类问题

- 将输入文本映射为所属类别（预定义的**封闭集合**）
- 最简单**最基本**的自然语言处理问题
- 应用场景
 - 垃圾邮件过滤、情感分类等
- 很多问题可以**转化**为文本分类问题





文本匹配问题

□ 判断**两段文本**之间的匹配关系

- 如复述关系、蕴含关系等

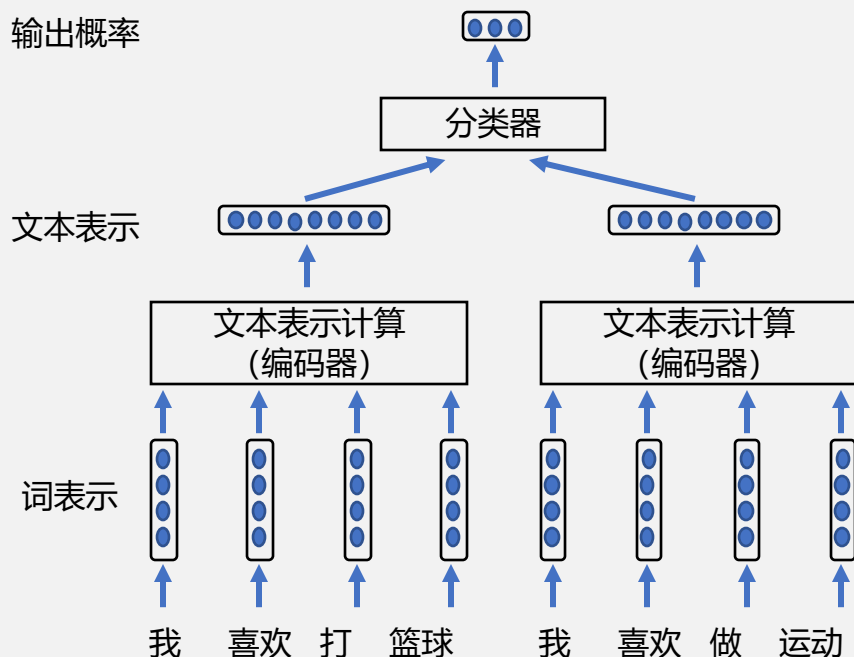
□ 解决方案

— **双塔**结构

- 两段文本分别通过两个模型映射为向量，然后判断两个向量之间的匹配关系

— **单塔**结构

- 将两段文本直接拼接，然后进行匹配关系分类



双塔结构示意图



结构预测问题

□ 输出类别之间具有较强的**相互关联性**

□ 自然语言处理的**本质**问题

□ 三种典型的结构预测问题

– 序列标注

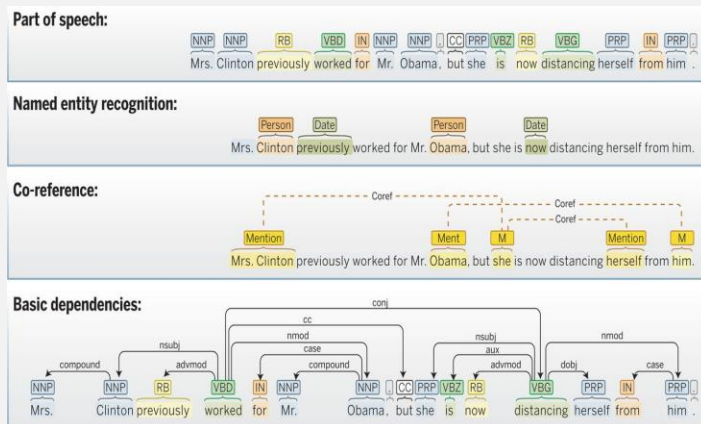
- 为输入文本序列中的每个词标注相应的标签
- 如词性标注：他/PN 喜欢/VV 下/VV 象棋/NN

– 序列分割

- 在文本序列中切分出子序列
- 可以转化为序列标注问题

– 图结构生成

- 输入自然语言，输出以图表示的结构



Julia Hirschberg and Christopher D. Manning.
Advances in Natural Language Processing. **Science** 2015



输入	我	爱	北	京	天	安	门	。
分词输出	B	B	B	I	B	I	I	B
命名实体输出	O	O	B-LOC	I-LOC	I-LOC	I-LOC	I-LOC	O

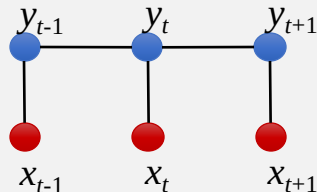


结构预测问题：序列标注

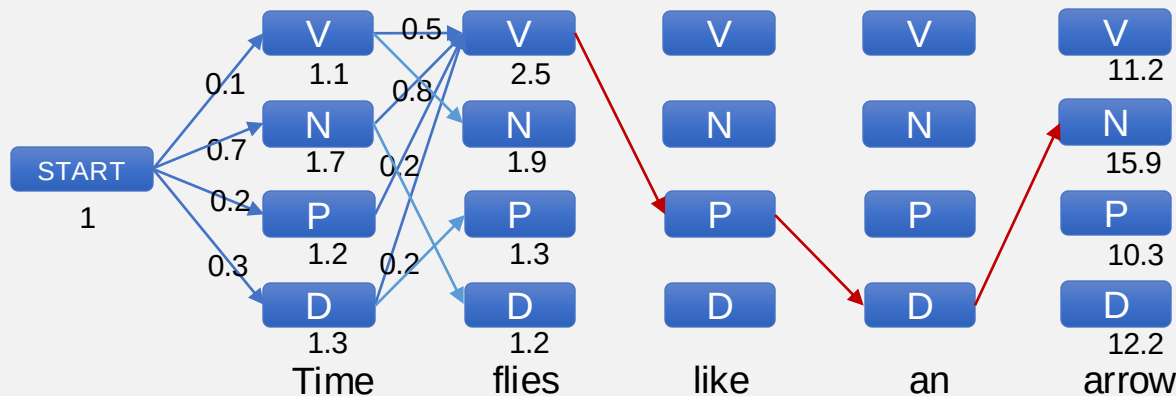
经典的序列标注模型

— 条件随机场 (Conditional Random Field, CRF)

$$\text{CRF } P(y_{[1:n]}|x_{[1:n]}) \propto \frac{1}{Z_{y_{[1:n]}}} \prod_{t=1}^n \exp \left(\begin{array}{l} \sum_j \lambda_j f_j(y_t, y_{t-1}) \\ + \sum_k \mu_k g_k(y_t, x_t) \end{array} \right)$$



维特比 (Viterbi) 解码算法





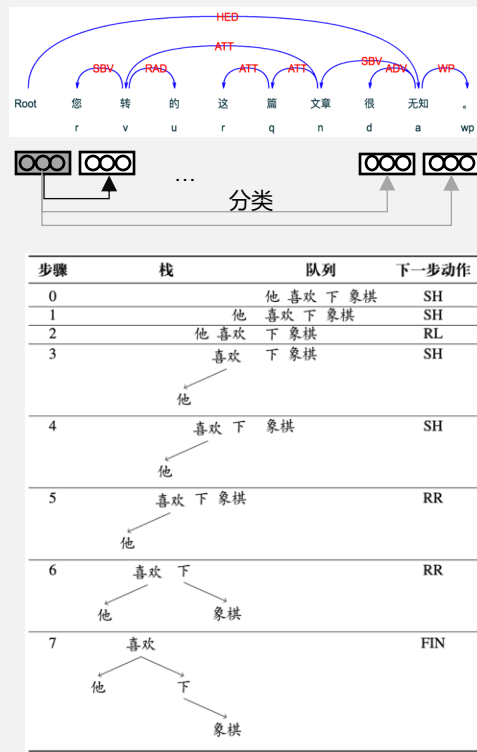
结构预测问题：图结构生成

□ 基于图的算法

- 计算图中任意两个节点之间连边的分数（类别）
- 根据解码算法生成图（树）结构

□ 基于转移的算法

- 将图结构的构建过程转化为状态转移序列
 - 如标准弧转移算法
 - 转移状态由一个栈和一个队列构成
 - 三种转移动作
 - 移进 (SH)、左弧 (RL) 和右弧 (RR)
- 对旧状态执行一个移动作转换为新状态
- 转移动作的选择本质上也是分类问题



面向依存句法分析的标准弧转移算法示例



序列到序列问题

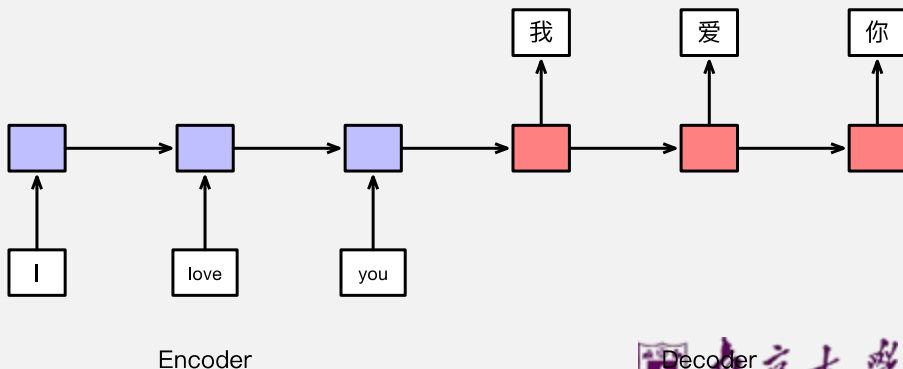
□ 将输入序列转换为输出序列

- 输入和输出的序列**不要求等长**，也**不要求词表一致**
- 泛化为“编码器—解码器”模型（Encoder-Decoder）
 - 本质上也是**分类问题**

□ 典型任务

任务	输入	输出
机器翻译	源语言	目标语言
文本摘要	原文	摘要
回复生成	用户语句	机器回复
图片描述生成	图片	文本描述
语音识别	语音	转写文本

“编码器—解码器”模型示例





大纲

- 文本的表示
- 自然语言处理任务
- 自然语言处理的基本问题
- 自然语言处理的评价指标
- 基础工具集与常用数据集



自然语言处理的评价

□ 准确率（Accuracy）

- 最简单直观的评价指标，常被用于文本分类、词性标注等问题

$$ACC^{cls} = \frac{\text{正确分类的文本数}}{\text{测试文本总数}}$$

$$ACC^{pos} = \frac{\text{正确标注的词数}}{\text{测试文本中词的总数}}$$

□ F值

- 针对某一类别的评价

$$F \text{ 值} = \frac{(\beta^2 + 1)PR}{\beta^2(P + R)}$$

$$P = \frac{\text{正确识别的命名实体数目}}{\text{识别出的命名实体总数}}$$

$$R = \frac{\text{正确识别的命名实体数目}}{\text{测试文本中命名实体的总数}}$$

- β 是加权调和参数； P 是精确率（Precision）； R 是召回率（Recall）
- 当权重为 $\beta = 1$ 时，表示精确率和召回率同样重要，也称F1值



自然语言处理的评价

□ 依存分析的评价

- UAS (Unlabeled Attachment Score)
 - 词的父节点被正确识别的准确率
- LAS (Labeled Attachment Score)
 - 词的父节点以及与父节点的句法关系都被正确识别的准确率

□ 机器翻译的评价

- BLEU (BiLingual Evaluation Understudy) 值
 - 统计机器译文与参考译文（可以不只有一个）中N-gram匹配的数目占机器译文中所有N-gram总数的比率

□ 对话系统的评价

- 由于回复的开放性，没有标准答案，很难自动评价
- 由于对话的交互性，不能简单地通过一轮人机对话就对系统进行评价
- 目前往往采用人工评价（流畅度、相关度、准确性等等）





大纲

- 文本的表示
- 自然语言处理任务
- 自然语言处理的基本问题
- 自然语言处理的评价指标
- 基础工具集与常用数据集



基础工具集与常用数据

基础工具集

- NLTK, TextBlob
- spaCy, Textacy
- LTP, CoreNLP
- PyTorch, PyTorch-NLP

常用数据集

- Wikipedia
- Common Crawl





NLTK

❑ Natural Language Toolkit

- <https://www.nltk.org/>

❑ 多种语料库和词典资源

- 生文本、PennTreebank样例
- WordNet

❑ 基本的自然语言处理工具集

- 分句
- 标记解析
- 词性标注
- 命名实体识别
- 句法分析



<https://data-flair.training/blogs/nltk-python-tutorial/>



TextBlob

□ TextBlob简介

- 一个用于处理文本数据的 Python 库，它提供了许多自然语言处理（NLP）功能。TextBlob 简化了文本处理过程，适合用于快速原型开发和实验。它基于 NLTK 和 Pattern 等库，提供了简洁的 API（<https://github.com/sloria/TextBlob>）
- TextBlob 主要适用于入门级的 NLP 项目，特别是处理英文文本时，它的简单易用性让许多开发者快速上手。

□ 帮助开发者进行常见的 NLP 任务

- **情感分析 (Sentiment Analysis)**：分析文本的情感倾向（积极、消极或中性）。
- **词性标注 (POS tagging)**：标注文本中的每个单词的词性（名词、动词等）。
- **名词短语提取 (Noun Phrase Extraction)**：从文本中提取名词短语。
- **翻译和语言检测**：支持通过 Google Translate API 实现文本翻译，检测文本的语言。
- **拼写纠正 (Spelling Correction)**：通过提供建议来纠正拼写错误。
- **词干提取 (Stemming) 和词形还原 (Lemmatization)**：将词语还原为其基本形式。





□ spaCy简介

- 一个用于自然语言处理的开源库，第一个面向商业受众的现代自然语言处理库，支持超过73+种语言，并为25种语言提供了训练代码。基于自然处理领域的最新研究，**spaCy提供了一系列高效且易用的工具**，用于文本预处理、文本解析、命名实体识别、词性标注、句法分析和文本分类等任务。（<https://spacy.io>）

□ 主要特点包括：

- **高性能**：SpaCy 是用 Cython 编写的，具有非常高的性能，尤其适用于处理大规模数据。
- **预训练模型**：提供了多种语言的预训练模型，包括英语、德语、西班牙语、法语等，可以直接用于词性标注、命名实体识别（NER）、依赖句法分析等任务。
- **支持现代NLP任务**：包括词向量、文本分类、分词、命名实体识别（NER）、依赖解析、文本相似度计算等。
- **易用性**：其 API 设计简单直观，适合快速开发和实验。
- **集成深度学习**：可以与深度学习库如 TensorFlow 和 PyTorch 配合使用，进一步增强其功能。



□ TextaCy简介

- 一个用于文本处理和自然语言处理 (NLP) 的 Python 库，旨在简化常见的文本清洗、处理和分析任务。它**建立在 spaCy 库的基础上**，因此具备了高效的 NLP 功能，同时增加了一些额外的工具来支持更复杂的文本分析。（<https://github.com/chartbeat-labs/textacy>）
- 主要特点是集成了丰富的文本分析功能，同时与 spaCy 等 NLP 工具兼容，方便快速处理大量文本数据。

□ 提供了许多实用的功能，包括：

- **文本清洗和预处理**：例如去除停用词、标点符号、特殊字符，进行分词、词干化、词形还原等。
- **特征提取**：支持从文本中提取关键词、短语、主题等。
- **文档相似性计算**：可以用来计算不同文本之间的相似性，常用于文本聚类 and 分类。
- **文本分析**：可以进行实体识别、关系提取、命名实体识别 (NER)、依存句法分析等。
- **主题建模**：支持使用 LDA (Latent Dirichlet Allocation) 等方法进行主题建模。



□ 语言技术平台 (Language Technology Platform, LTP)

- <http://ltp.ai>
- 高效、高精度**中文自然语言处理**基础平台
- 中文词法、句法、语义分析等6项自然语言处理核心技术



□ 2020年7月发布4.0版本

- 基于预训练模型
- 多任务学习机制

□ 安装

- \$ pip install ltp

在线演示
直观了解语言技术平台能为你做什么

国务院总理李克强调研上海外高桥时提出，支持上海积极探索新机制。

句子视图 篇章视图 XML视图

☒ 词性标注 ☒ 命名实体 ☒ 句法分析 ☒ 语义角色标注 ☒ 语义依存分析

段落1句子1: 国务院总理李克强调研上海外高桥时提出，支持上海积极探索新机制。

标签释义

Tag	关系类型	Description
Agnt	施事关系	Agent
Dir	趋向角色	Direction
ePurp	目的关系	event Purpose
Feat	描写角色	Description
Mann	方式角色	Manner
mPunc	标点标记	Punctuation Marker
mTime	时间标记	Time
Nmod	名字修饰角色	Name-modifier
Prod	成事关系	Product
Root	根节点	Root



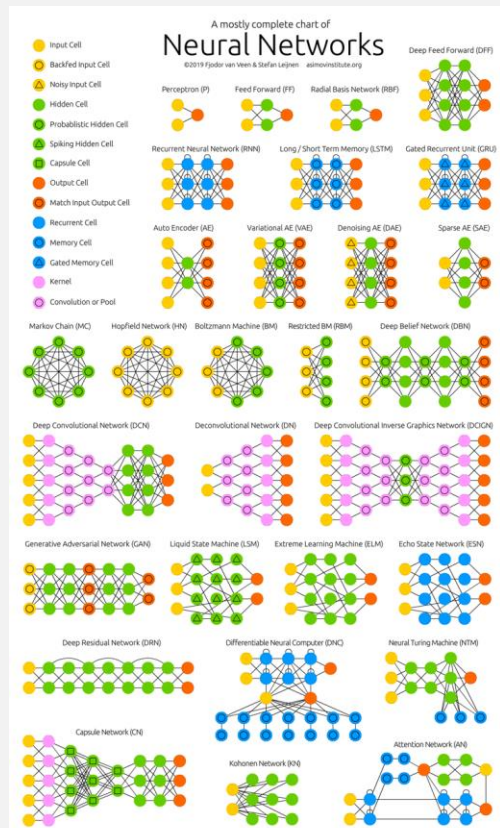
□ CoreNLP 简介

- CoreNLP是**Java**中进行自然语言处理的一站式工具。
- 可以**通过命令行或其 Web 服务与 CoreNLP 进行交互**；许多人在编写自己的 JavaScript、Python 或其他语言的代码时使用 CoreNLP。
- CoreNLP使用户能够为文本生成语言学标注，**包括标记和句子边界、词性、命名实体、数字和时间值、依存句法和成分句法分析、指代消解、情感分析、引用归属和关系。**
- 目前，CoreNLP支持**8种语言**：阿拉伯语、中文、英语、法语、德语、匈牙利语、意大利语和西班牙语。



PyTorch

- ❑ Facebook开发的开源深度学习库
 - <https://pytorch.org/>
- ❑ 基于张量 (Tensor) 的**数学运算**工具包
 - GPU加速的张量计算
 - 自动进行**微分**计算
- ❑ PyTorch的优势
 - 框架简洁
 - 入门简单, 容易上手
 - 支持动态神经网络构建
 - 与Python语言无缝结合
 - Debug方便





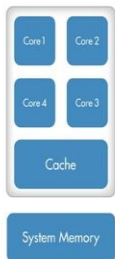
□ PyTorch-NLP 简介

- PyTorch-NLP 是一个用于自然语言处理（NLP）的 Python 库。
- 是考虑到最新的研究成果构建的，并且从一开始就旨在支持快速原型开发。
- PyTorch-NLP 提供了预训练的词向量、采样器、数据集加载器、评估指标、神经网络模块和文本编码器。

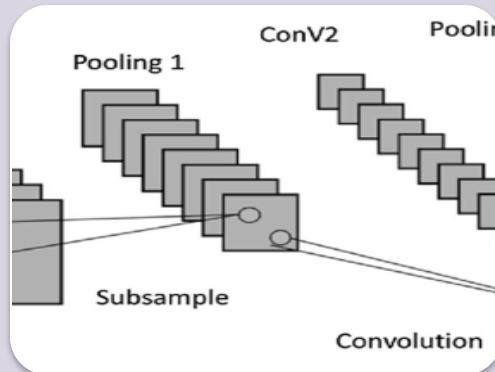
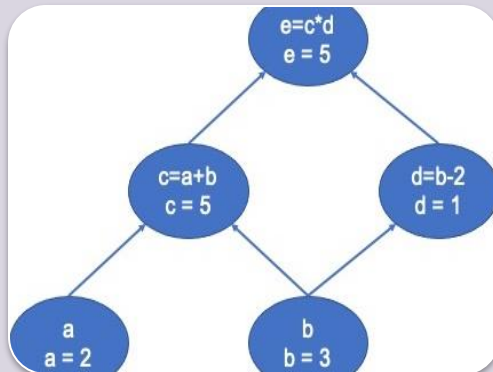
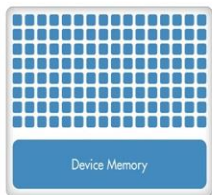


PyTorch 三要素

CPU (Multiple Cores)



GPU (Hundreds of Cores)



张量 (Tensor)

- 数据存储
- 支持GPU

表达式 (Expression)

- 数据运算
- 自动微分计算

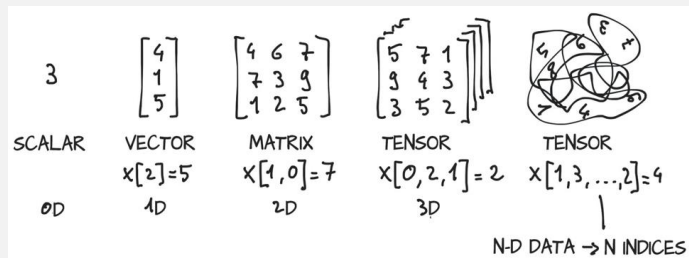
模块 (Module)

- 神经网络层
- 支持自定义

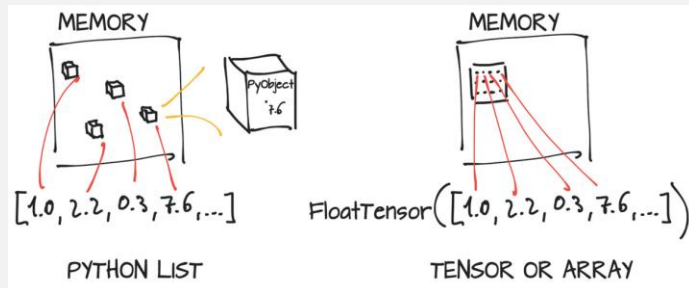


张量 (Tensor)

□ 又称为多维数组

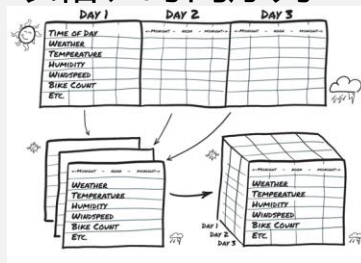


□ 与Python列表的区别

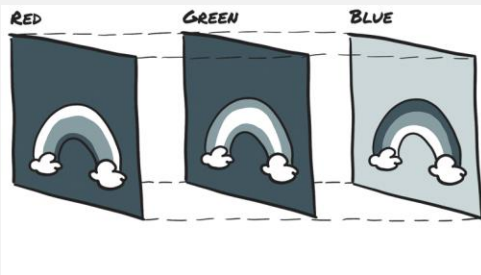


□ 可表示现实世界中的各种数据

— 表格、时间序列



— (彩色) 图像



`batch = torch.zeros(100, 3, 256, 256, dtype=torch.uint8)`



PyTorch作为张量库

□ 支持各种张量操作（类似NumPy）

- 创建张量
- 索引、切片

$$M * M * M$$

$$M \in \mathbb{R}^{1000 \times 1000}$$

□ 支持GPU

- 快!

Numpy

```
In [2]: M = numpy.random.randn(1000,1000)

In [3]: timeit -n 500 M.dot(M).dot(M)
500 loops, best of 3: 30.7 ms per loop
```

PyTorch

```
In [4]: N = torch.randn(1000,1000).cuda()

In [5]: timeit -n 500 N.mm(N).mm(N)
500 loops, best of 3: 474 μs per loop
```





表达式 (Expression)

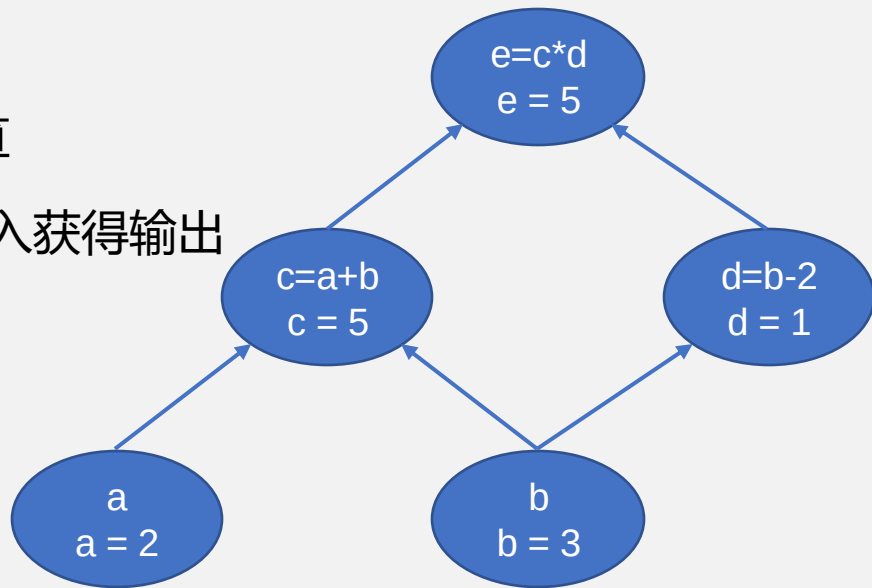
□ 通过计算图描述表达式

— 如: $e = (a + b) * (b - 2)$

□ 计算图中每个节点执行一个运算

— 前向运算 (Forward) : 根据输入获得输出

```
>>> a = torch.tensor([2.])
>>> b = torch.tensor([3.])
>>> c = a + b
>>> d = b - 2
>>> e = c * d
>>> print(c, d, e)
tensor([5.], grad_fn=<AddBackward0>)
tensor([1.], grad_fn=<SubBackward0>)
tensor([5.], grad_fn=<MulBackward0>)
```





表达式 (Expression)

□ 反向传播 (Back-propagation)

— 计算输出对每个输入的导数

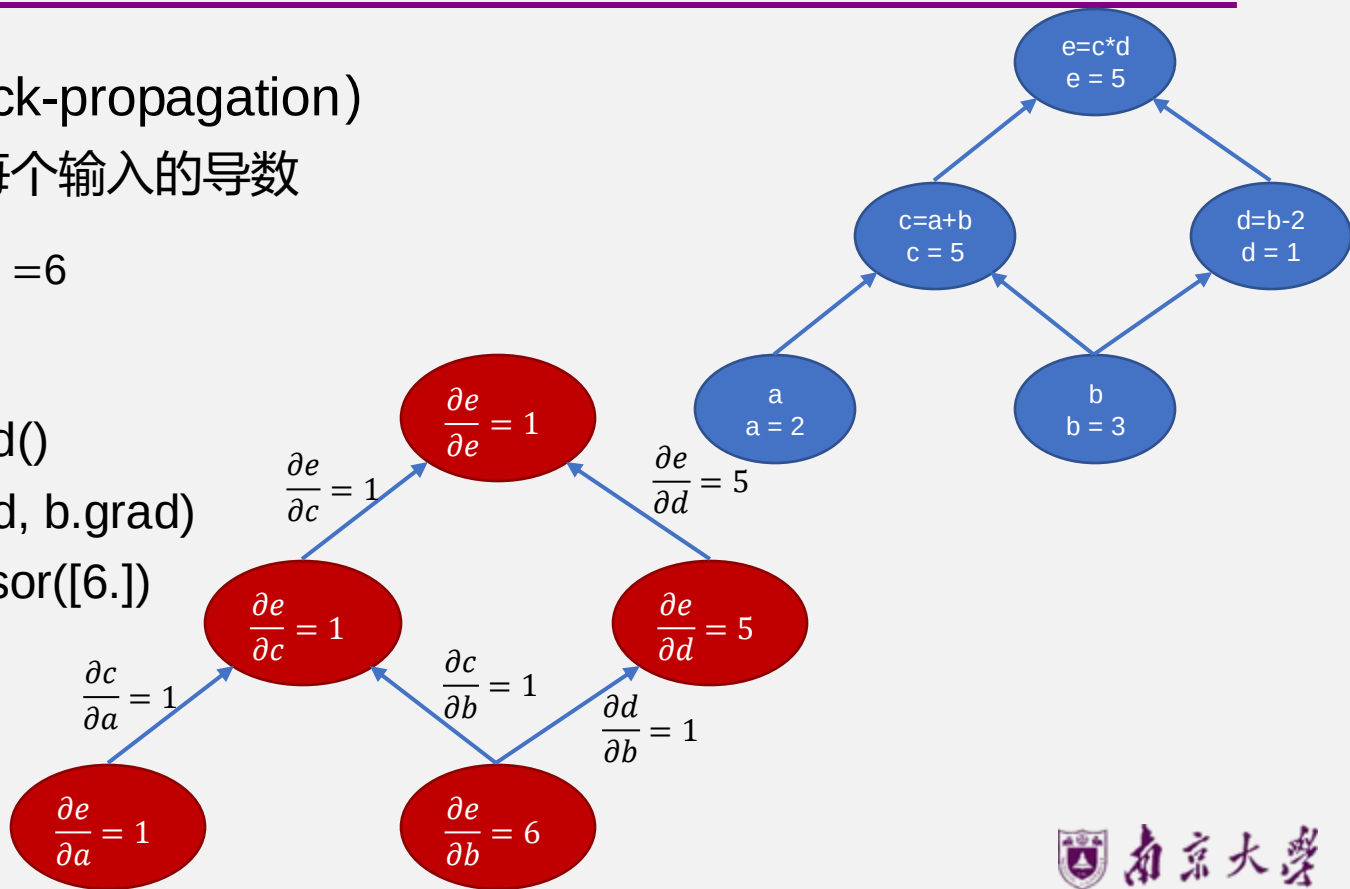
$$\bullet \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial b} = 6$$

□ 自动微分计算

```
>>> e.backward()
```

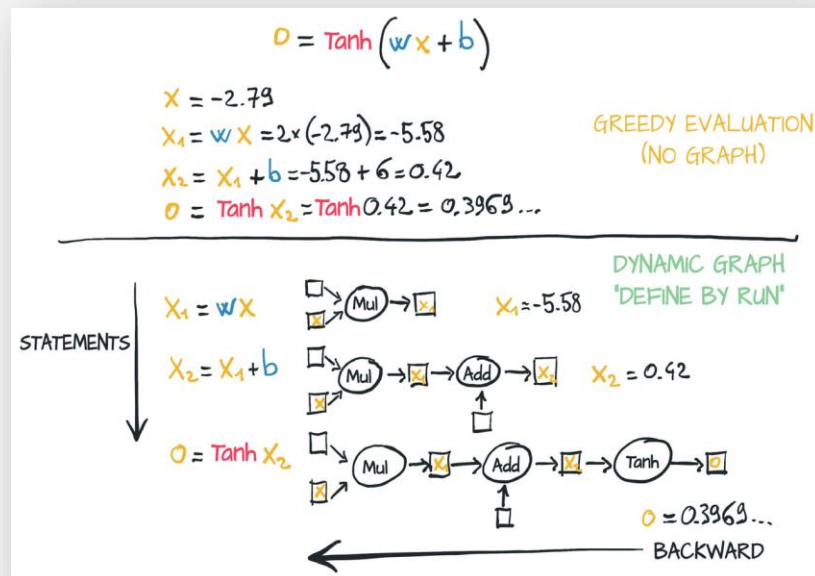
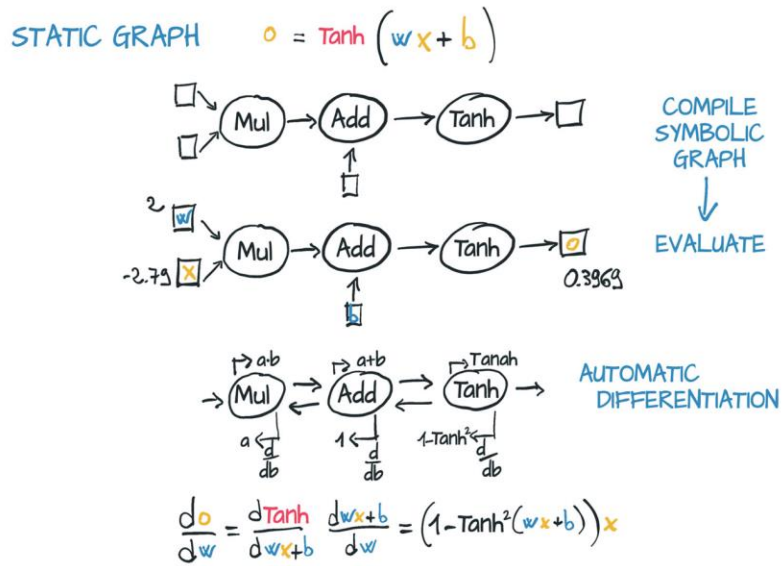
```
>>> print(a.grad, b.grad)
```

```
tensor([1.]) tensor([6.])
```





静态图与动态图





模块 (Module)

包名	功能	描述
torch	Tesnor / Expression	类似NumPy的张量库，支持GPU以及自动微分
torch.nn	Module	灵活的神经网络库，提供多种神经网络层
torch.optim	Module	多种优化算法
torch.utils	Module	数据集、数据加载等辅助工具



大规模预训练数据集

□ 维基百科 (Wikipedia)

- 快照 (2020年10月23日) <https://dumps.wikimedia.org/>

文件名	内容	大小/MB
zhwiki-latest-abstract.xml.gz	所有词条摘要	≈ 147
zhwiki-latest-all-titles.gz	所有词条标题	≈ 33
zhwiki-latest-page.sql.gz	所有词条标题及摘要	≈ 204
zhwiki-latest-pagelinks.sql.gz	所有词条外链	≈ 890
zhwiki-latest-pages-articles.xml.bz2	所有词条正文	≈ 1,952

– 处理流程

- 纯文本抽取 → 中文繁简转换 → 数据清洗

□ Common Crawl数据 (<https://commoncrawl.org/>)

- 大规模网络爬虫数据集
- 使用Facebook的CC-Net工具下载和处理
 - https://github.com/facebookresearch/cc_net



小结

- 文本的表示
- 自然语言处理任务
- 自然语言处理的基本问题
- 自然语言处理的评价指标
- 基础工具集与常用数据集